

Detailed Windows Booting Process

The Windows boot process is the sequence of operations that begins when you press the power button and ends when the Windows desktop becomes ready.

A simplified sequence is:

Power ON → CPU Reset → UEFI/BIOS → POST → Boot Device → Windows Boot Manager → Windows Loader → Kernel → Drivers → System Processes → Services → Login → Windows Desktop

1. Power ON

When you press the computer's power button, the computer does not immediately start Windows.

First, electrical power is supplied to the different components.

The main components involved are:

CPU — executes instructions.

RAM — temporarily stores instructions and data being used.

SSD/HDD — permanently stores Windows and other files.

Motherboard — connects the components.

UEFI/BIOS firmware — contains the initial instructions needed to start the computer.

What happens to the CPU?

Initially, the CPU is held in a reset state. Once the power and hardware conditions are suitable, the CPU is released from reset.

The CPU then begins executing instructions from a predefined location associated with the system firmware.

At this point, Windows is not running yet.

The CPU is initially running the motherboard's firmware rather than the Windows operating system.

2. CPU Starts UEFI/BIOS Firmware

The first software that the CPU executes is the motherboard's firmware.

Modern computers generally use UEFI, while older computers commonly use traditional BIOS.

UEFI is stored in non-volatile memory on the motherboard, so it remains available even when the computer is powered off.

The basic relationship is:

Power ON



CPU starts executing



CPU executes UEFI firmware

UEFI provides the initial environment required to initialize hardware and locate an operating system.

Why doesn't the CPU directly start Windows?

Because the CPU needs instructions telling it:

Where Windows is stored

Which storage device contains Windows

How to initialize the hardware

Where to find the Windows boot files

UEFI performs these initial tasks.

3. POST — Power-On Self-Test

One of the early firmware operations is POST (Power-On Self-Test).

POST checks whether important hardware is functioning sufficiently for the computer to continue booting.

The firmware checks or initializes things such as:

CPU

RAM

Storage controllers

Keyboard

Display hardware

Basic motherboard devices

For example, the firmware needs to determine whether usable RAM is available.

If there is a serious hardware problem, the computer may:

Display an error message

Produce beep codes on some systems

Display a diagnostic code

Stop the boot process

Example

Suppose the computer has no usable RAM.

The CPU cannot simply continue loading Windows because Windows requires RAM to execute.

Therefore:

Power ON



CPU starts firmware



Firmware checks RAM



RAM problem detected



Boot may stop

If the basic hardware is working, the firmware continues.

4. Hardware Initialization

After the initial hardware checks, UEFI initializes important hardware sufficiently for the boot process.

For example, it may initialize:

RAM

CPU-related hardware

Storage controllers

USB controllers

Display output

PCIe devices

Network hardware, where applicable

The purpose is to create an environment in which the firmware can locate and execute the operating-system bootloader.

Important point

The firmware does not fully initialize Windows.

It performs enough hardware initialization for the system to proceed to the operating system.

Later, Windows loads its own drivers and performs additional hardware initialization.

5. UEFI Finds the Boot Device

Now UEFI needs to find out where the operating system is located.

The computer may have several storage devices:

SSD 1

SSD 2

HDD

USB drive

Network boot source

UEFI uses its configured boot order to decide which source to try.

For a normal Windows installation, the boot process commonly looks like:

UEFI



EFI System Partition



Windows Boot Manager

The EFI System Partition (ESP) is a special partition used by UEFI systems.

It contains boot-related files, including the Windows boot manager.

6. Windows Boot Manager

UEFI loads and executes the Windows Boot Manager.

On a typical UEFI installation, the Windows Boot Manager is associated with:

bootmgfw.efi

The .efi indicates that this is a UEFI executable.

The CPU now stops executing the relevant firmware boot code and begins executing the Windows Boot Manager code loaded into memory.

Conceptually:

UEFI



loads Windows Boot Manager



CPU executes Boot Manager

What does Boot Manager do?

The Windows Boot Manager determines how Windows should be started.

It uses the Boot Configuration Data (BCD).

The BCD contains configuration information related to booting Windows.

It can contain information such as:

Available Windows installations

Default operating system

Boot options

Recovery options

Debugging options

Boot parameters

7. Boot Configuration Data — BCD

The BCD can be thought of as a database containing Windows boot configuration information.

For a normal computer with one Windows installation, you may never see it.

However, it becomes more noticeable when:

Multiple Windows installations exist

Advanced boot options are configured

Windows recovery is required

Troubleshooting options are used

For example:

Windows Boot Manager



BCD



Which Windows installation?



Windows 11

The Boot Manager uses this information to determine what should happen next.

8. Windows OS Loader

After determining which Windows installation should be started, Boot Manager starts the Windows operating-system loader.

On UEFI systems, this is commonly:

winload.efi

The Windows loader has a very important job.

It prepares the Windows kernel and other essential components for execution.

It loads important components from the Windows installation into RAM.

Among these are:

Windows kernel

Hardware Abstraction Layer

Boot-start drivers

Other essential boot components

Conceptually:

SSD



Windows system files



Windows Loader



RAM



CPU executes them

9. Loading the Windows Kernel

One of the most important steps is loading the Windows kernel.

The Windows kernel is the central component of the operating system.

A major kernel executable is:

ntoskrnl.exe

The kernel manages many fundamental aspects of the computer, including:

CPU scheduling

Memory management

Processes

Threads

Hardware interaction

Interrupt handling

Security mechanisms

Device management

Why is the kernel important?

Applications cannot directly control all hardware resources themselves.

Instead, they interact with operating-system facilities, which are ultimately managed by the kernel and other Windows components.

For example:

Application



Windows APIs



Operating system



Kernel



Hardware

10. Loading the Hardware Abstraction Layer

Windows also loads the Hardware Abstraction Layer (HAL).

The HAL provides an abstraction between parts of Windows and the underlying hardware platform.

This allows Windows to work across many different hardware configurations without every operating-system component needing to understand every hardware-specific detail.

Simplified:

Windows components



HAL



Hardware

The HAL works closely with the kernel and hardware-related components.

11. Loading Boot-Critical Drivers

Windows also loads drivers required during the early boot process.

A driver is software that allows the operating system to communicate with a particular type of hardware.

Examples include drivers associated with:

Storage controllers

File systems

Certain bus devices

Other hardware required during startup

This is particularly important because Windows needs to access the storage device containing the rest of the operating system.

For example:

Windows Kernel



Storage driver



Storage controller



SSD

Without the necessary storage support, Windows may not be able to access the files it needs.

12. CPU Begins Executing the Windows Kernel

At this point, control transitions from the bootloader to the Windows kernel.

The CPU begins executing kernel instructions.

What does “CPU begins executing the Windows kernel” mean?

When the computer starts, the **CPU executes the bootloader's instructions first.**

The **bootloader** prepares the system and loads the important parts of Windows, including the **Windows kernel**, into memory.

Once the bootloader has finished its main job, it **transfers control to the Windows kernel.**

This is a major milestone:

UEFI



Boot Manager



Windows Loader



Windows Kernel

The operating system is now taking control of the computer.

13. Kernel Initialization

The Windows kernel performs extensive initialization.

It establishes the mechanisms needed to manage the computer.

Some important areas include:

CPU management

Windows needs to manage the available CPU cores.

For example:

CPU

├— Core 0

├— Core 1

├— Core 2

└— Core 3

The operating system creates the mechanisms necessary to schedule threads on available processors.

The Windows kernel decides:

Which thread will run on which CPU core and when?

(कौन सा थ्रेड किस CPU core पर और कब चलेगा?)

For example:

Core 0 → Thread A

Core 1 → Thread B

Core 2 → Thread C

Core 3 → Thread D

After a little while (Thodi der baad):

Core 0 → Thread E

Core 1 → Thread F

Core 2 → Thread A

Core 3 → Thread C

This is called decision-making thread scheduling.

So the simple meaning of the whole paragraph: During Windows Kernel startup, it sets up the necessary systems to manage the CPU and its cores. Then it decides when and how different threads should run on the available CPU cores.

(तो पूरे पैराग्राफ का सरल मतलब ये है: विंडोज़ कर्नेल के स्टार्टअप के दौरान, यह CPU और उसके कोर्स को मैनेज करने के लिए जरूरी सिस्टम सेट करता है। फिर यह तय करता है कि अलग-अलग थ्रेड्स को उपलब्ध CPU कोर्स पर कब और कैसे चलना चाहिए।)

Memory management

Windows establishes its memory-management system.

It manages:

Physical RAM

Virtual memory

Memory protection

Address spaces

Paging

Process management

Windows establishes the mechanisms needed to create and manage processes.

A process is a running program together with its associated resources.

Interrupt handling

Hardware devices need a way to notify the CPU that something requires attention.

The operating system configures mechanisms for handling hardware and software interrupts.

14. Memory Management During Boot

RAM is extremely important during boot.

Windows cannot simply execute everything directly from the SSD.

Instead, required code and data are brought into memory.

For example:

SSD

|

| Read Windows files

↓

RAM

|

| Instructions/data

↓

CPU

|

| Execute

↓

Result

The CPU executes instructions primarily from memory rather than repeatedly executing Windows program files directly from the storage device.

SSD vs RAM

SSD:

Permanent storage

Retains data when power is removed

Stores Windows files

RAM:

Temporary working memory

Loses its contents when power is removed

Holds code and data currently being used

CPU:

Executes instructions

Performs calculations

Controls program execution

15. Device Driver Initialization

Windows begins initializing more hardware and drivers.

A driver acts as an interface between Windows and a hardware device.

For example:

Windows



GPU Driver



Graphics Hardware

Another example:

Windows



Storage Driver



NVMe Controller



SSD

Windows loads the appropriate drivers according to the system configuration.

This is why a computer can use hardware from many different manufacturers.

16. Session Manager — smss.exe

An important Windows system process called Session Manager Subsystem, commonly associated with:

smss.exe

plays a role during the transition into the normal Windows environment.

It initializes important parts of the system environment and helps start processes required for Windows sessions.

The exact internal boot sequence is complex and varies between Windows versions and configurations, so it should not be viewed as a simple fixed linear sequence of every process.

17. Windows System Processes Start

Additional fundamental Windows processes are started.

Examples include:

`csrss.exe`

Client/Server Runtime Subsystem.

It is a critical Windows system process involved in important user-mode system functionality.

`wininit.exe`

Windows Initialization process.

It helps initialize the Windows environment and start important system processes and services.

`services.exe`

Service Control Manager.

It manages Windows services.

`lsass.exe`

Local Security Authority Subsystem Service.

It performs important security-related functions, including authentication and security-policy enforcement.

These processes work together to establish the Windows operating environment.

18. Windows Services Start

Windows contains many services that operate in the background.

Examples can include services associated with:

Networking

Windows Update

Security

Audio

Device management

Event logging

Background tasks

The Service Control Manager manages these services.

A simplified representation is:

Windows Kernel



System Processes



Service Control Manager



Windows Services



Background functionality

Not every service necessarily starts at exactly the same time. Startup behavior depends on the service configuration and Windows version.

19. User Interface Is Initialized

Windows then initializes more of the graphical environment.

This includes components needed to display the Windows login experience.

The computer now has enough of the operating system running to interact with the user.

You may see:

Windows logo



Loading



Login screen

20. Login Process

The user enters authentication information.

For example:

Password

PIN

Fingerprint

Other configured authentication methods

Windows verifies the authentication information using its security infrastructure.

If authentication succeeds, Windows creates or resumes the user's session.

Conceptually:

User



Enter credentials



Windows authentication



Credentials verified



User session created

21. User Session Starts

Once authentication is successful, Windows prepares the user's environment.

Windows loads user-specific information and starts applications/components configured to run when the user logs in.

These can include:

User settings

Desktop configuration

Startup applications

User environment variables

Background applications

22. Windows Explorer Starts

A key component of the normal Windows graphical environment is:

explorer.exe

Windows Explorer provides much of the familiar Windows shell.

It is responsible for functionality associated with:

Desktop

Taskbar

Start menu

File Explorer

Desktop icons

This is why the appearance of the desktop generally means the Windows shell has become available.

23. Desktop Appears

Finally, the user sees the Windows desktop.

simplified final sequence is:

Authentication



User Session



Windows Shell



explorer.exe



Desktop



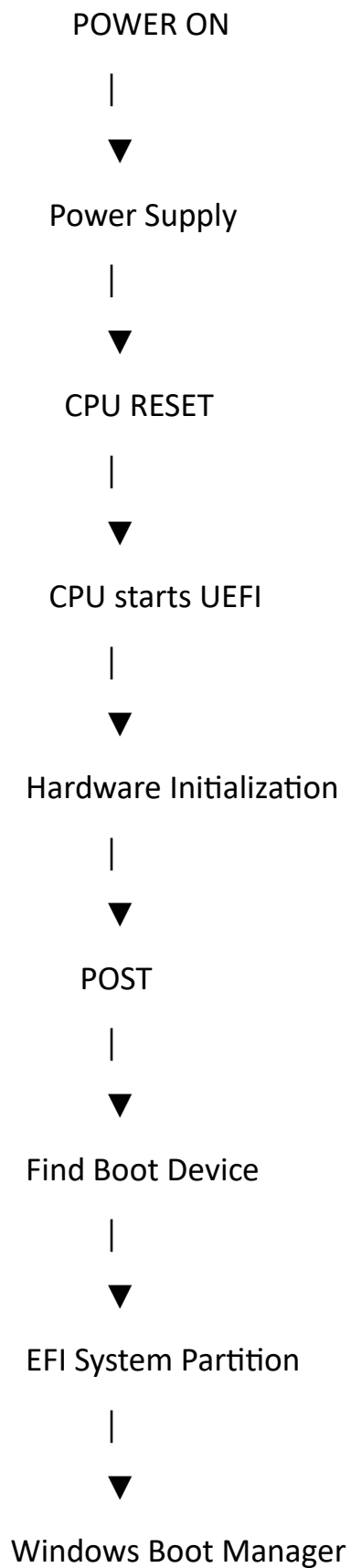
Taskbar + Start Menu + Icons

At this point, the computer is ready for normal use.

However, booting is not necessarily finished in the sense that every background task has completed. Some services and startup applications may continue initializing after the desktop becomes visible.

24. Complete Boot Process

Putting everything together:



(bootmgfw.efi)



Read Boot Configuration

(BCD)



Windows OS Loader

(winload.efi)



Load Kernel + HAL + Drivers



Windows Kernel

(ntoskrnl.exe)



Kernel Initialization



Hardware/Drivers Setup



System Processes



Windows Services



Login Screen



User Authentication



User Session



Windows Shell

(explorer.exe)



DESKTOP



Windows Ready

25. Role of CPU, RAM, SSD and UEFI

The easiest way to understand the whole process is to look at the responsibilities of each component.

Component Main role during boot

CPU Executes the instructions

RAM Holds instructions and data currently being used

SSD/HDD Permanently stores Windows files

UEFI Initializes hardware and finds the bootloader

Boot Manager Determines how Windows should be started

Windows Loader Loads the kernel and essential boot components

Kernel Takes control and manages system resources

Drivers Allow Windows to communicate with hardware

Services Provide background system functionality

Explorer Provides much of the Windows graphical shell

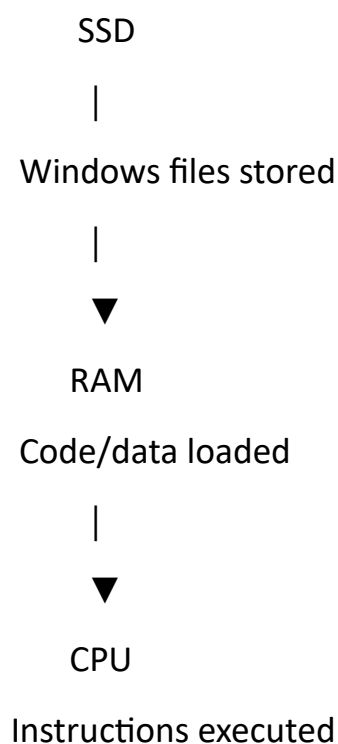
26. The Most Important Concept: CPU Executes, Storage Stores

A common misunderstanding is that the CPU "loads Windows."

More precisely:

The storage device stores Windows. The CPU executes Windows instructions after those instructions are loaded into memory.

For example:





Windows operating

So the three components work together:

SSD → RAM → CPU

The SSD provides persistent storage, RAM provides fast working memory, and the CPU executes the instructions.

27. One Simple Real-World Analogy

Imagine a company opening in the morning.

SSD = filing room — contains all the company's documents.

RAM = desk — holds the documents currently being worked on.

CPU = employee — reads and processes the information.

UEFI = receptionist — gets the building ready and directs the employee to the correct department.

Boot Manager = supervisor — decides which operating system/work environment should start.

Windows Kernel = manager — manages the company's resources.

Drivers = translators — allow the manager to communicate with different departments/equipment.

Services = background staff — perform ongoing tasks.

Explorer = front office — provides the interface the user interacts with.

Thus, when you press the power button, the computer goes from a powered-off state → firmware → bootloader → operating system → user interface.